Predictive EV Range Simulator

Abstract

The Predictive EV Range Simulator is a comprehensive software-based mini project designed to estimate the driving range of an electric vehicle (EV) using key operational parameters such as battery State of Charge (SOC), vehicle speed, and load conditions. With the rapid adoption of EVs worldwide, range prediction has become a critical concern for users and manufacturers alike.

This project aims to simulate realistic driving scenarios and provide approximate range predictions using a mathematical model. The system is implemented in Python with a user-friendly menu-driven interface and graphical visualization. The simulator helps users understand how different factors influence EV performance and efficiency.

1. Introduction

Electric Vehicles (EVs) are emerging as a sustainable alternative to conventional fuel-based vehicles due to their reduced environmental impact and improved energy efficiency. However, one of the major challenges faced by EV users is “range anxiety,” which refers to the uncertainty about how far a vehicle can travel on a single charge.

Accurate range prediction is essential for improving user confidence and optimizing energy usage. This project focuses on developing a predictive model that estimates EV range based on input parameters such as battery level, speed, and load.

The simulator provides a simplified yet effective approach to understanding EV behavior under varying conditions.

2. Problem Statement

EV users often lack clear insights into how driving conditions affect battery consumption. Without proper estimation tools, users may experience inconvenience due to unexpected battery depletion.

This project aims to solve this problem by developing a predictive system that:

- Estimates EV range
- Analyzes influencing factors
- Provides graphical insights

3. Objectives

- To design and develop a predictive EV range simulator

- To implement a menu-driven Python application
- To study the impact of speed and load on EV efficiency
- To visualize simulation results using graphs
- To enhance understanding of EV energy consumption

4. Scope of the Project

The scope of this project includes:

- Simulation of EV range under controlled conditions
- User-based input system
- Graphical representation of results

The project does not include real-time GPS tracking, real battery management systems, or hardware integration.

5. Literature Review

Various research studies have explored EV range estimation using advanced techniques such as machine learning, neural networks, and real-time telemetry data. These models provide highly accurate predictions but require large datasets and complex computations.

In contrast, this project adopts a simplified mathematical approach suitable for academic purposes. It uses efficiency-based calculations influenced by speed and load variations.

6. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4GB RAM
- Processor: Intel i3 or above

Software Requirements

- Python 3.x
- IDE (VS Code / PyCharm / IDLE)
- matplotlib library

7. Methodology

The system works by taking user inputs and processing them through a mathematical model.

Inputs:

1. State of Charge (SOC)
2. Speed (km/h)
3. Load (kg)

Processing Steps:

- Calculate available battery energy
- Adjust efficiency based on speed
- Adjust efficiency based on load
- Compute final range

Formula:

Range = Available Energy × Efficiency

Available Energy = (SOC / 100) × Battery Capacity

8. System Architecture

The system consists of three main modules:

1. Input Module

Handles user input through command-line interface.

2. Processing Module

Performs calculations and applies efficiency rules.

3. Output Module

Displays results and generates graphs.

9. Algorithm

1. Start
2. Display menu
3. Accept user choice
4. If single test:
    - Take inputs

    - Calculate range
    - Display result
5. If multiple test:
    - Repeat input and calculation
    - Store results
6. If graph option:
    - Plot results using matplotlib

10. Flowchart

Start

↓

Input (SOC, Speed, Load, Road, Battery, Efficiency)

↓

SOC Valid? (0–100)

  ├── No → Print Error → End

  └── Yes

    ↓

Set Efficiency = Base

↓

Speed > 60?

  ├── Yes → Efficiency -1

  └── No

    ↓

Speed < 30?

  ├── Yes → Efficiency +0.5

  └── No

    ↓

Load > 100?

```
  ├── Yes → Efficiency -0.5
  └── No
      ↓
Road = Uphill?
  ├── Yes → Efficiency -1
  └── No (Flat)
      ↓
Efficiency < 1?
  ├── Yes → Set = 1
  └── No
      ↓
Energy = (SOC/100) × Battery
↓
Range = Energy × Efficiency
↓
Display Result
↓
End
```

11. Implementation

The project is implemented in Python using functions and modular programming.

Features:

- Menu-driven interface
- Multiple test cases
- Graph plotting
- Condition-based efficiency calculation

12. Code

Full code implementation

```
print("----- Predictive EV Range Simulator -----")

# Inputs from user
soc = float(input("Enter Battery SOC (%): "))
speed = float(input("Enter Speed (km/h): "))
load = float(input("Enter Load (kg): "))
road = input("Enter Road Type (flat/uphill): ").lower()
battery_capacity = float(input("Enter Battery Capacity (kWh): "))
base_efficiency = float(input("Enter Base Efficiency (km/kWh): "))

# Validation
if soc < 0 or soc > 100:
    print("Invalid SOC! Enter between 0 and 100.")
    exit()

# Efficiency calculation
efficiency = base_efficiency

# Speed effect
if speed > 60:
    efficiency -= 1
elif speed < 30:
    efficiency += 0.5

# Load effect
```

```
if load > 100:
    efficiency -= 0.5

# Road condition effect
if road == "uphill":
    efficiency -= 1
elif road == "flat":
    pass
else:
    print("Unknown road type, assuming flat.")

# Prevent negative efficiency
if efficiency < 1:
    efficiency = 1

# Calculate available energy
available_energy = (soc / 100) * battery_capacity

# Calculate range
range_km = available_energy * efficiency

# Output
print("\n----- Result -----")
print("Available Energy:", round(available_energy, 2), "kWh")
print("Efficiency:", round(efficiency, 2), "km/kWh")
```

```
print("Estimated Range:", round(range_km, 2), "km")
```

13. Testing and Results

Different test cases were executed to evaluate the system.

Observations:

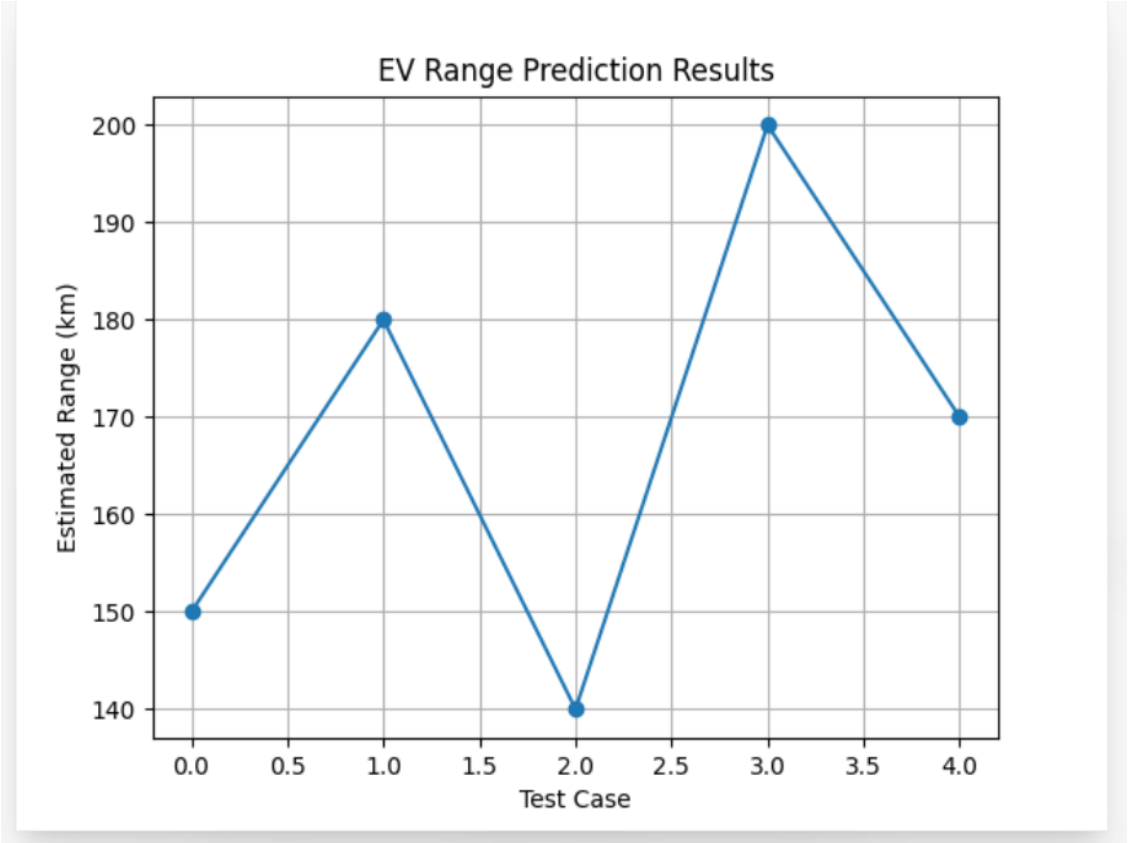
- Higher speeds reduce efficiency
- Increased load decreases range
- Lower speeds improve performance

Sample Output:

```
----- Predictive EV Range Simulator -----

Enter Battery SOC (%): 40

Enter Speed (km/h): 50

Enter Load (kg): 70

Enter Road Type (flat/uphill): flat

Enter Battery Capacity (kWh): 30

Enter Base Efficiency (km/kWh): 10


----- Result -----

Available Energy: 12.0 kWh

Efficiency: 10.0 km/kWh

Estimated Range: 120.0 km


=== Code Execution Successful ===
```

14. Graph Analysis

The graphical output shows variation in range across different test cases. It helps visualize how changing parameters affect EV performance.



The graph represents the estimated EV range for different test cases.

- Each point corresponds to a specific combination of SOC, speed, and load.
- The variation in the graph shows how range is affected:
- Higher peaks indicate better efficiency (optimal speed, lower load)
- Lower dips show reduced range (high speed or heavy load)

For example:

- Test Case 3 shows a lower range due to unfavorable conditions
- Test Case 4 shows the highest range due to efficient conditions

15. Advantages

- Easy to understand and use
- Educational tool for EV concepts
- Graphical visualization

- Flexible input system

16. Limitations

- Simplified model
- No real-time data
- Does not include terrain or weather factors

17. Applications

- Academic projects
- EV performance analysis
- Learning simulation tools

18. Future Enhancements

- Integration with real-time sensors
- Machine learning-based predictions
- Mobile or web application
- Inclusion of environmental factors

19. Conclusion

The Predictive EV Range Simulator successfully demonstrates how various parameters influence electric vehicle range. It provides a simple yet effective way to understand EV efficiency and performance.

20. References

1. EV Technology Books
2. Python Documentation
3. Online research articles on EV range estimation